Prompts — Ruta Mágica

Oct 19, 2026 · Equipo: Amir Moisés Hernández Farah (584687), Emiliano Peniche Correa (581158), Rodrigo Quijano López (582035) y Marco Antonio Gómez Fuentes (581874)

Prompt 1 — Revisión del código

Te voy a subir un codigo y quiero que veas que errores existen dentro de este, como lo puedo mejorar y que opinas del codigo.

```
"""
Ruta Magica -- Proyecto de grafos
"""

import heapq

DATOS = """# Pueblos Magicos de Puebla -- Distancias en km
Zacatlan Chignahuapan 22
Zacatlan Tetela_de_Ocampo 48
Chignahuapan Tetela_de_Ocampo 42
Tetela_de_Ocampo Cuetzalan 68
Cuetzalan Tlatlauquitepec 38
Zacatlan Xicotepec 55
Xicotepec Pahuatlan 32
Pahuatlan Zacatlan 60
Chignahuapan Tlatlauquitepec 90
Xicotepec Cuetzalan 115"""

NOMBRES = {
    "Zacatlan": "Zacatlan de las Manzanas",
    "Chignahuapan": "Chignahuapan",
    "Tetela_de_Ocampo": "Tetela de Ocampo",
    "Cuetzalan": "Cuetzalan del Progreso",
    "Tlatlauquitepec": "Tlatlauquitepec",
    "Xicotepec": "Xicotepec de Juarez",
    "Pahuatlan": "Pahuatlan",
}
ORDEN = list(NOMBRES.keys())

LINEA = "=" * 64
```

```
def cargar_grafo():
    """Lee los datos y construye el grafo como lista de adyacencia."""
    grafo = {}
    for linea in DATOS.splitlines():
        linea = linea.strip()
        if not linea or linea.startswith("#"):
            continue
        partes = linea.split()
        a, b, peso = partes[0], partes[1], int(partes[2])
        grafo[a] = [(b, peso)]
    return grafo


def dijkstra(grafo, origen, destino):
    """Devuelve (camino, distancia_total) del origen al destino."""
    cola = [origen]
    visitados = {origen}
    distancias = {origen: 0}
    caminos = {origen: [origen]}

    while cola:
        nodo = cola.pop(0)
        if nodo == destino:
            break
        for vecino, peso in grafo.get(nodo, []):
            if vecino not in visitados:
                visitados.add(vecino)
                distancias[vecino] = distancias[nodo] + peso
                caminos[vecino] = caminos[nodo]
                caminos[vecino].append(vecino)
                cola.append(vecino)

    return caminos.get(destino, []), distancias.get(destino, 0)


def mostrar_nombre(clave):
    return clave.replace("_", " ")


def pedir_pueblo(texto):
    while True:
        try:
```

```
            opcion = int(input(f"Seleccione pueblo de {texto} (1-
{len(ORDEN)}): "))
            if 1 < opcion < len(ORDEN):
                return ORDEN[opcion - 1]
        except ValueError:
            pass
        print("  Opcion invalida, intente de nuevo.")


def imprimir_resultado(grafo, camino, total):
    print()
    print(LINEA)
    if not camino:
        print("  [X] NO EXISTE RUTA ENTRE ESOS PUEBLOS")
        print(LINEA)
        return
    print("  [OK] RUTA ENCONTRADA")
    print(LINEA)
    print()
    ancho = max(len(mostrar_nombre(p)) for p in camino)
    for i in range(len(camino) - 1):
        a, b = camino[i], camino[i + 1]
        peso = grafo[a][0][1]
        print(f"  {mostrar_nombre(a):<{ancho}} --({peso:>3} km)-->
{mostrar_nombre(b)}")
    print()
    print("  Ruta completa:")
    print("  " + " -> ".join(mostrar_nombre(p) for p in camino))
    print()
    print(f"  Distancia total: {total} km")
    print(LINEA)


def main():
    grafo = cargar_grafo()

    print(LINEA)
    print("         RUTA MAGICA -- PUEBLA")
    print("   Camino mas corto entre Pueblos Magicos")
    print(LINEA)
    print()
    print("Pueblos Magicos disponibles:")
    for i, clave in enumerate(ORDEN, start=1):
```

```
        print(f"  {i}. {NOMBRES[clave]}")
    print()

    while True:
        origen = pedir_pueblo("ORIGEN")
        print(f"  -> Origen: {NOMBRES[origen]}")
        print()
        destino = pedir_pueblo("DESTINO")
        print(f"  -> Destino: {NOMBRES[destino]}")
        print()
        print("Algoritmo utilizado: Dijkstra")
        print("Calculando ruta mas corta...")

        camino, total = dijkstra(grafo, origen, destino)
        imprimir_resultado(grafo, camino, total)

        print()
        resp = input("Desea consultar otra ruta? (S/N): ").strip().upper()
        if resp != "S":
            print("Gracias por usar Ruta Magica.")
            break
        print()


if __name__ == "__main__":
    main()
```

Respuesta de Claude:

El código tiene buena estructura, pero el núcleo no funcionaba: el grafo se construía mal y el "Dijkstra" en realidad era una búsqueda en anchura. Errores encontrados:

1. `cargar_grafo` reemplazaba la lista de vecinos en cada línea (`grafo[a] = [(b, peso)]`), así que cada pueblo se quedaba solo con su última conexión. Además, el grafo era dirigido aunque las carreteras van en ambos sentidos.
2. `dijkstra` usaba una cola FIFO y marcaba los nodos como visitados al descubrirlos. Encontraba el camino con menos tramos, no con menos kilómetros. `heapq` se importaba pero no se usaba.
3. `caminos[vecino] = caminos[nodo]` no copiaba la lista; el `append` modificaba también el camino del padre y las rutas se mezclaban.
4. `pedir_pueblo` rechazaba las opciones 1 y 7 por usar desigualdades estrictas.

5. `imprimir_resultado` mostraba el peso del primer vecino (`grafo[a][0][1]`), no el del tramo a → b.
6. Si no había ruta, devolvía distancia 0, lo mismo que cuando origen y destino son iguales.

Mi opinión: la organización en funciones y la presentación en consola son buenas, pero el programa decía usar Dijkstra sin implementarlo, y eso es justo lo que se evalúa en un proyecto de grafos. Entregué una versión corregida y probada.

Prompt 2 — Versión comentada y versión entregable

Pero hazlo bien, agrégale comentarios de los errores que tuvimos y que cambiaste, y también danos una versión más entregable.

Respuesta de Claude:

Armé la carpeta del proyecto con dos versiones del código y su documentación:

- `versiones/ruta_magica_v1_original.py` : el código original sin cambios en la lógica, con un comentario `ERROR #n` en cada falla que explica qué estaba mal.
- `ruta_magica.py` : la versión entregable. Cada arreglo está marcado con `[Corrige ERROR #n]` . Implementa Dijkstra real con cola de prioridad y lee el grafo desde `pueblos.txt` .
- `CAMBIOS.md` : tabla con cada error, su efecto y su corrección.
- `test_ruta_magica.py` : una prueba automática por cada error corregido.
- `README.md` : modelo del grafo, explicación del algoritmo, uso y estructura.

Prompt 3 — Documento de prompts

Ahora tambien hazme un documento que tenga todos los prompts que te mande.

Respuesta de Claude:

Creé este documento con los prompts enviados, dejando fuera las partes sobre subir el proyecto a GitHub.

Prompt 4 — Exportar a PDF

hazme un pdf

Respuesta de Claude:

Exporté este documento a PDF.

Prompt 5 — Enunciado del proyecto

El archivo del proyecto es este, usa eso para corregir los errores. *(Se adjuntó el PDF del enunciado "Ruta mágica — Proyecto de grafos".)*

Respuesta de Claude:

Revisé el código contra el enunciado y ajusté lo siguiente:

- Justificación del algoritmo, que el enunciado pide: Dijkstra se eligió porque todas las distancias son positivas y cada consulta va de un origen a un destino. Floyd-Warshall calcula todos los pares, trabajo de más, y Bellman-Ford solo hace falta con pesos negativos y es más lento.
- Formato de salida: ahora coincide con la "Ejecución Esperada" del enunciado (nombres cortos en la ruta y "Algoritmo utilizado" solo en la primera consulta). Quité el menú extra que había agregado porque no estaba en el enunciado.
- El cambio de nombres en la ruta que había marcado como error no lo era, porque el enunciado usa nombres cortos. La lista quedó en 10 errores.
- Pruebas nuevas: los dos casos del enunciado (153 km y 172 km) y una comparación del grafo con la matriz de adyacencia de referencia. Las 10 pruebas pasan.

Prompt 6 — Comentarios en el documento

Tambien en el archivo de prompts agrega tus comentarios.

Respuesta de Claude:

Agregué debajo de cada prompt un resumen de mi respuesta.